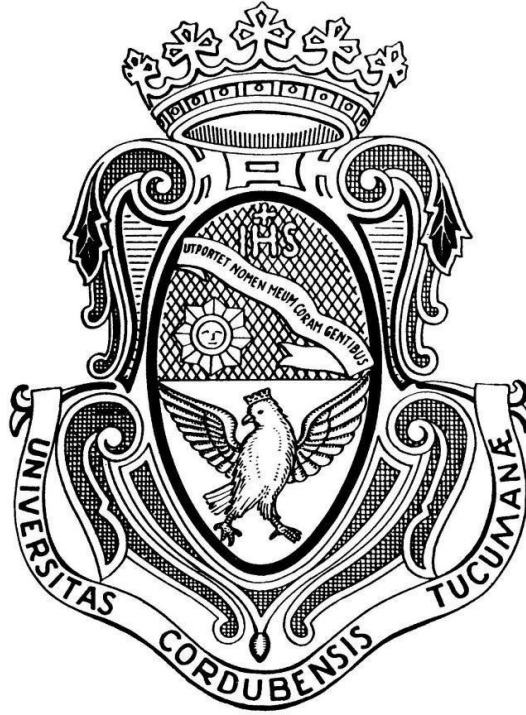


Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Sistemas de Computación

Trabajo Práctico N° 5 - Device Drivers

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

12 de junio de 2026

Índice

1	Preparación	2
2	Character Device Driver	4
2.1	Dispositivo	4
2.2	Aplicación de Lectura	6
2.3	Servidor	7
3	Conclusiones	9
4	Link al Repositorio	10

1. Preparación

Primero se instalaron las herramientas necesarias para la compilación cruzada:

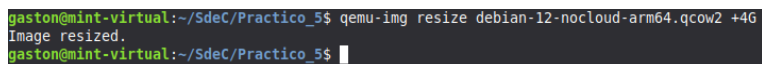
```
sudo apt install -y build-essential git bc bison flex libssl-dev \  
gcc-aarch64-linux-gnu qemu-system-arm qemu-utils \  
qemu-efi-aarch64
```

Después, se descarga una imagen Debian 12 nocloud arm64 mediante el siguiente link (debian-12-nocloud-arm64.qcow2):

<https://cloud.debian.org/images/cloud/bookworm/latest/>

Se utiliza la extensión “.qcow2 ” porque es el formato de disco virtual nativo y por defecto de QEMU, entonces es más compatible. La versión “nocloud” indica que la imagen está empaquetada para correr de forma local (offline). Y por último, arm64 es la arquitectura necesaria porque se quiere simular una Raspberry Pi.

Luego se ajusta el tamaño de la imagen a 4 GB (es un requisito de QEMU que el tamaño de la imagen tiene que ser potencia de 2):



```
gaston@mint-virtual:~/SdeC/Practico_5$ qemu-img resize debian-12-nocloud-arm64.qcow2 +4G  
Image resized.  
gaston@mint-virtual:~/SdeC/Practico_5$
```

Figura 1: Ajuste del tamaño de la imagen.

Ahora se puede arrancar la imagen mediante los siguientes comandos:

```
# Ejecuta el binario principal de QEMU para emular un sistema arm64  
qemu-system-aarch64 \  

```

```
# Usa una placa base virtual genérica (virt), simula el procesador ARM Cortex-A72, asigna  
# 2 núcleos de procesamiento y otorga 2 GB de RAM  
-M virt -cpu cortex-a72 -smp 2 -m 2G \  

```

```
# Carga un archivo que actúa como UEFI  
-bios /usr/share/qemu-efi-aarch64/QEMU_EFI.fd \  

```

```
# Conecta una unidad de almacenamiento, utiliza ‘virtio’ como interfaz, aclara el tipo  
# de archivo y el archivo en sí  
-drive if=virtioi,format=qcow2,file=debian-12-nocloud-arm64.qcow2 \  

```

```
# Se configura para simular una red usando el espacio de usuario, dándole conexión a la VM.  
# Se le pone nombre a la configuración y se crea una regla de redirección de puertos  
# (desde 5022 en el host hacia el 22 de la VM, que es el de ssh)  
-netdev user,id=net0,hostfwd=tcp::5022-:22 \  

```

```
# Se instala una tarjeta de red a la placa de la VM y se la conecta con la configuración definida anter  
-device virtio-net-pci,netdev=net0 \  

```

```
# No se utiliza la salida de video simulada de QEMU  
-nographic
```

En la figura 2 se observa que se inicia correctamente y aparece un prompt de login, que se tiene que llenar con root porque es la configuración por defecto.

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
[ OK ] Reached target timers.target - Timer Units.
[ OK ] Listening on dbus.socket-Bus System Message Bus Socket.
[ OK ] Listening on uidd.socketm - UUID daemon activation socket.
[ OK ] Reached target sockets.target - Socket Units.
[ OK ] Reached target basic.target - Basic System.
Starting dbus.service - D-Bus System Message Bus...
Starting e2scrub_reap.service ext4 Metadata Check Snapshots...
Starting systemd-logind.service - User Login Management...
Starting systemd-user-sessions.service - Permit User Sessions...
[ OK ] Started dbus.service - D-Bus System Message Bus.
[ OK ] Finished systemd-user-sessions.service - Permit User Sessions.
[ OK ] Started getty@tty1.service - Getty on tty1.
[ OK ] Started serial-getty@ttyAMA0.service - Serial Getty on ttyAMA0.
[ OK ] Reached target getty.target - Login Prompts.
[ OK ] Started systemd-logind.service - User Login Management.
[ OK ] Started unattended-upgrade.timer - Unattended Upgrades Shutdown.
[ OK ] Finished e2scrub_reap.service ext4 Metadata Check Snapshots.
[ OK ] Reached target multi-user.target - Multi-User System.
[ OK ] Reached target graphical.target - Graphical Interface.
Starting systemd-update-utmp.service - Record Runlevel Change in UTMP...

Debian GNU/Linux 12 localhost ttyAMA0
localhost login: 
```

Figura 2: Inicio de la simulación.

Ahora se configura la contraseña:

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
[ OK ] Reached target getty.target - Login Prompts.
[ OK ] Started systemd-logind.service - User Login Management.
[ OK ] Started unattended-upgrade.timer - Unattended Upgrades Shutdown.
[ OK ] Finished e2scrub_reap.service ext4 Metadata Check Snapshots.
[ OK ] Reached target multi-user.target - Multi-User System.
[ OK ] Reached target graphical.target - Graphical Interface.
Starting systemd-update-utmp.service - Record Runlevel Change in UTMP...

Debian GNU/Linux 12 localhost ttyAMA0

localhost login: root
Linux localhost 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@localhost:~# passwd
New password:
Retype new password:
passwd: password updated successfully
root@localhost:~# 
```

Figura 3: Se configura la contraseña.

Se ejecutan los siguientes comandos para configurar SSH:

```
# Se instala el servidor SSH
apt update && apt install -y openssh-server sudo build-essential

# Utiliza el Stream Editor para habilitar el acceso SSH para el usuario Root
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config

# Se reinicia el SSH para que se apliquen los cambios
systemctl restart ssh
```

Una vez ejecutados los comandos, se puede hacer la conexión mediante ssh y la verificación del kernel:

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_5$ ssh -p 5022 root@localhost
The authenticity of host '[localhost]:5022 ([127.0.0.1]:5022)' can't be established.
ED25519 key fingerprint is SHA256:ESQ4j8ZC5DSQ9049e+jdkQQtNQ05fHg9dFn4U2v3Qc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:5022' (ED25519) to the list of known hosts.
root@localhost's password:
Linux localhost 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Jun 11 22:16:56 2026
root@localhost:~# uname -r
6.1.0-49-arm64
root@localhost:~# uname -m
aarch64
root@localhost:~#
    
```

Figura 4: Conexión mediante ssh.

Se instalan los headers del kernel en la VM mediante el siguiente comando:

```
apt install -y linux-headers-$(uname -r)
```

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_5$ ssh -p 5022 root@localhost
The authenticity of host '[localhost]:5022 ([127.0.0.1]:5022)' can't be established.
ED25519 key fingerprint is SHA256:ESQ4j8ZC5DSQ9049e+jdkQQtNQ05fHg9dFn4U2v3Qc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:5022' (ED25519) to the list of known hosts.
root@localhost's password:
Linux localhost 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Jun 11 22:16:56 2026
root@localhost:~# uname -r
6.1.0-49-arm64
root@localhost:~# uname -m
aarch64
root@localhost:~#
    
```

Figura 5: Conexión mediante ssh.

2. Character Device Driver

2.1. Dispositivo

Explicación:

El archivo seniales.c funciona de la siguiente forma:

```
sample_cb(struct timer_list *t)
```

Se utiliza esta función para generar las señales que luego se quieren medir. Para la señal senoidal se utiliza una tabla con valores de seno precalculados (24 valores), recorriendo la tabla para generar la señal.

Para la señal cuadrada se va aumentando un contador que cuando vale menos de la mitad del periodo entonces se pone en alto la señal, mientras que si el contador vale mas de la mitad del periodo se pone la señal en bajo, logrando una señal cuadrada.

Esta función se ejecuta cada 1 segundo mediante mod_timer y jiffies.

En la interfaz ed usuario, se utilizan las funciones de file operations (fops) para interactuar con el espacio de usuario:

```
my_read(struct file *file, char __user *ubuf, size_t count, loff_t *ppos)
```

Esta función lee la señal elegida, el tiempo transcurrido y el valor actual de la señal mediante spinlocks para que no se modifiquen los valores mientras se los utiliza. Luego empaqueta los datos en una cadena de texto y los transfiere mediante la función del kernel “simple_read_from_buffer” hacia el espacio de memoria de usuario.

```
my_write(struct file *file, const char __user *ubuf, size_t count, loff_t *ppos)
```

Esta función se usa para que el usuario mande datos hacia el dispositivo (por ejemplo mediante comandos echo). Se utiliza para cambiar qué señal se quiere visualizar.

Las funciones “my_open” y “my_release” solamente se usan para indicar que se abrió o cerró el archivo /dev/seniales.

```
sdec_init(void)
```

Esta función se ejecuta cuando se carga el módulo y realiza 5 pasos:

- alloc_chrdev_region le pide al kernel que asigne los números Major y Minor para el dispositivo
- cdev_init y cdev_add enlazan el dispositivo interno del kernel con la estructura de operaciones fops.
- class_create crea una categoría para el dispositivo de sistema de archivos “sysfs”
- device_create instruye al sistema udev para que cree el archivo de nodo /dev/seniales
- timer_setup y mod_timer arrancan el temporizador para que genere datos

```
sdec_exit(void)
```

Esta función se ejecuta cuando se descarga el módulo, y su trabajo consiste en revertir todo lo hecho por la función init, por lo que destruye el nodo del dispositivo, la clase, y libera los números de identificación.

Implementación:

Primero se pasan los archivos seniales.c y Makefile mediante el comando scp:

```
gaston@mint-virtual:~/Sdec/Practico_5$ scp -P 5022 seniales.c Makefile root@localhost:~/driver/
root@localhost's password:
seniales.c          100% 4553   206.0KB/s   00:00
Makefile            100% 171    11.5KB/s   00:00
gaston@mint-virtual:~/Sdec/Practico_5$
```

Figura 6: Transferencia del archivo seniales.c y Makefile.

Se compila el módulo:

```
root@localhost:~# cd ~/driver
root@localhost:~/driver# ls -l
total 12
-rw-r--r-- 1 root root 171 Jun 12 06:28 Makefile
-rw-r--r-- 1 root root 4553 Jun 12 06:28 seniales.c
root@localhost:~/driver# make
make -C /lib/modules/6.1.0-49-arm64/build M=/root/driver/modules
make[1]: Entering directory '/usr/src/linux-headers-6.1.0-49-arm64'
  CC [M] /root/driver/seniales.o
  MODPOST /root/driver/Module.symvers
  CC [M] /root/driver/seniales.mod.o
  LD [M] /root/driver/seniales.ko
  BTF [M] /root/driver/seniales.ko
Skipping BTF generation for /root/driver/seniales.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.1.0-49-arm64'
root@localhost:~/driver#
```

Figura 7: Compilación del módulo seniales.ko.

Ahora se verifica la compilación, se carga el módulo y se comprueba que se cargó correctamente:

```
root@localhost:~/driver# ls
Makefile      modules.order  seniales.ko    seniales.mod.c  seniales.o
Module.symvers  seniales.c      seniales.mod  seniales.mod.o
root@localhost:~/driver# insmod seniales.ko
root@localhost:~/driver# ls -l /dev/seniales
crw-rw-rw- 1 root root 237, 0 Jun 12 06:37 /dev/seniales
```

Figura 8: Carga del módulo seniales.ko.

También se verifica con los comandos cat y dmesg:

```
root@localhost:~/driver# cat /proc/devices | grep seniales
237 seniales
root@localhost:~/driver# dmesg | tail -n 5
[ 1533.165127] mimodulo: loading out-of-tree module taints kernel.
[ 1533.171311] mimodulo: module verification failed: signature and/or required key missing - tainting kernel
[ 1533.186289] Módulo cargado en el kernel.
[ 1608.661112] Módulo descargado del kernel.
[ 3661.079218] seniales: CARGADO. major=237 minor=0 -> /dev/seniales
root@localhost:~/driver#
```

Figura 9: Verificación del módulo.

Ahora se realiza una prueba de funcionamiento:

```
root@localhost:~/driver# cat /dev/seniales
1 313 0
root@localhost:~/driver# echo -n 2 > /dev/seniales
root@localhost:~/driver# cat /dev/seniales
2 365 1000
root@localhost:~/driver# cat /dev/seniales
2 385 1000
root@localhost:~/driver# cat /dev/seniales
2 402 1000
root@localhost:~/driver# cat /dev/seniales
2 404 1000
root@localhost:~/driver# cat /dev/seniales
2 406 0
root@localhost:~/driver# cat /dev/seniales
2 425 1000
root@localhost:~/driver#
```

Figura 10: Testeo del módulo.

2.2. Aplicación de Lectura

Explicación:

El archivo reader.c funciona de la siguiente forma:

En las funciones `read_sample` y `select_signal`, el hardware virtual (el Character Device) se trata exactamente como si fuera un archivo de texto plano. Al abrir el dispositivo y usar operaciones estándar del sistema (`read` y `write`), el programa desencadena las funciones fops (File Operations) que viven dentro del módulo del kernel para interactuar directamente con el sensor simulado.

Función `select()`: Esta función resuelve el problema de escuchar al teclado y al sensor al mismo tiempo configurando un temporizador de 1 segundo. Funciona con una doble condición: si un usuario presiona una tecla antes de que pase ese segundo, interrumpe la espera y usa `select_signal` para enviar el comando de cambio de canal. Si el segundo transcurre sin que el usuario toque nada, la función expira y el programa avanza para leer una nueva muestra del dispositivo.

Función `scale()`: Esta función identifica la señal actual, le asigna una unidad (en este caso Voltios) y aplica una regla de tres para mapear los valores “crudos” del kernel (que van de 0 a 1000) a tensiones realistas: un máximo de 3.3V para la señal cuadrada y 5.0V para la senoidal.

Por último, los datos procesados no se imprimen como texto convencional, sino que se formatean como una estructura JSON utilizando `printf`. Esto permite que la salida estándar del programa pueda ser capturada y analizada fácilmente por el script de Python que levanta el servidor web.

Implementación: Acá se realiza la compilación cruzada y se pasa el archivo compilado reader mediante el comando scp:

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_5$ aarch64-linux-gnu-gcc -O2 -Wall -static -o reader reader.c
gaston@mint-virtual:~/SdeC/Practico_5$ file reader
reader: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=dbd89e2a9f96f49b000d9462e6cf7d70e4468fdd, for GNU/Linux 3.7.0, not stripped
gaston@mint-virtual:~/SdeC/Practico_5$ scp -P 5022 reader root@localhost:~/driver/
root@localhost's password:
reader                                100% 753KB 917.0KB/s 00:00
gaston@mint-virtual:~/SdeC/Practico_5$
```

Figura 11: Compilación cruzada y transferencia de archivo reader.

Se ejecuta el archivo reader mediante el siguiente comando:

```
./reader 1 /dev/seniales
```

El resultado se observa en la figura 12:

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_5
File Edit View Terminal Tabs Help
[ 1302.233250] seniales: release()
[ 1302.593430] seniales: open()
[ 1302.593935] seniales: senial seleccionada = 2
[ 1302.595549] seniales: release()
[ 1302.608351] seniales: open()
[ 1302.612959] seniales: release()
[ 1303.622003] seniales: open()
[ 1303.623736] seniales: release()
[ 1304.632481] seniales: open()
[ 1304.633706] seniales: release()
[ 1305.648220] seniales: open()
[ 1305.648612] seniales: release()
[ 1306.654507] seniales: open()
[ 1306.660867] seniales: release()
[ 1306.855248] seniales: open()
[ 1306.855605] seniales: senial seleccionada = 1
[ 1306.859796] seniales: release()
[ 1306.862052] seniales: open()
[ 1306.866294] seniales: release()
[ 1307.879989] seniales: open()
[ 1307.884912] seniales: release()
[ 1308.894839] seniales: open()
[ 1308.900048] seniales: release()

{"senial":1,"type":"senoidal","unit":"V","t":256,"raw":707,"value":3.535}
{"senial":1,"type":"senoidal","unit":"V","t":257,"raw":866,"value":4.330}
{"senial":1,"type":"senoidal","unit":"V","t":258,"raw":966,"value":4.830}
{"senial":1,"type":"senoidal","unit":"V","t":259,"raw":1000,"value":5.000}
{"senial":1,"type":"senoidal","unit":"V","t":260,"raw":966,"value":4.830}
{"senial":1,"type":"senoidal","unit":"V","t":261,"raw":866,"value":4.330}
{"senial":1,"type":"senoidal","unit":"V","t":262,"raw":707,"value":3.535}
{"senial":1,"type":"senoidal","unit":"V","t":263,"raw":500,"value":2.500}
{"senial":1,"type":"senoidal","unit":"V","t":264,"raw":259,"value":1.295}
{"senial":1,"type":"senoidal","unit":"V","t":265,"raw":0,"value":0.000}
{"senial":1,"type":"senoidal","unit":"V","t":266,"raw":259,"value":1.295}
{"senial":1,"type":"senoidal","unit":"V","t":267,"raw":500,"value":2.500}
{"senial":1,"type":"senoidal","unit":"V","t":268,"raw":707,"value":3.535}
2
{"senial":2,"type":"cuadrada","unit":"V","t":268,"raw":0,"value":0.000}
{"senial":2,"type":"cuadrada","unit":"V","t":269,"raw":0,"value":0.000}
{"senial":2,"type":"cuadrada","unit":"V","t":270,"raw":0,"value":0.000}
{"senial":2,"type":"cuadrada","unit":"V","t":271,"raw":1000,"value":3.300}
{"senial":2,"type":"cuadrada","unit":"V","t":272,"raw":1000,"value":3.300}
1
{"senial":1,"type":"senoidal","unit":"V","t":272,"raw":966,"value":4.830}
{"senial":1,"type":"senoidal","unit":"V","t":273,"raw":866,"value":4.330}
{"senial":1,"type":"senoidal","unit":"V","t":274,"raw":707,"value":3.535}
```

Figura 12: Testeo del archivo reader.

2.3. Servidor

Explicación del archivo servidor.py:

La función del archivo servidor.py es ejecutar el programa reader.c y crear un servidor web para poder ver los datos en un navegador.

Mediante subprocess.Popen, el script ejecuta el programa en C (reader) en segundo plano, captura su salida estándar (stdout) a través de tuberías para leer continuamente los datos JSON que este genera, y utiliza la entrada estándar (stdin) para inyectarle comandos, actuando como un puente completamente automático sin necesidad de una terminal visible.

El archivo almacena las últimas lecturas en una cola rápida de doble extremo (deque) configurada con un límite de 120 elementos, por lo que borra automáticamente los datos más viejos al llenarse. Además, utiliza un cerrojo de sincronización (state_lock) para evitar condiciones de carrera, garantizando que el hilo que lee el sensor no modifique la lista de puntos en el exacto milisegundo en que el servidor web intenta enviarlos al navegador.

Servidor Web Integrado (HTTP Handler): Implementa un servidor ligero (`ThreadingHTTPServer`) que define cómo responder a las peticiones HTTP del navegador. Sirve el archivo visual `index.html` en la ruta raíz (`/`), expone los datos acumulados del sensor en tiempo real empaquetados en JSON a través de `/data`, y recibe los comandos de los botones de la interfaz a través de `/select` para ordenar el cambio de canal.

Sincronización de Gráficos (Epoch): Utiliza una variable contadora llamada `epoch` que se incrementa cada vez que el usuario cambia de señal. Cuando el frontend de la web consulta el endpoint `/data` y nota que este número cambió, la lógica de la interfaz sabe instantáneamente que debe limpiar el lienzo (`canvas`) y empezar a dibujar la nueva curva desde cero.

Explicación del archivo `index.html`:

Motor Gráfico (Chart.js): Utiliza esta librería externa de JavaScript para crear el lienzo (`canvas`). Se encarga matemáticamente de dibujar la cuadrícula, los ejes, y conectar los puntos para formar la línea de la onda.

Consulta Continua (Polling): Mediante la instrucción `setInterval(tick, 1000)`, la página web le pregunta al servidor Python (`fetch('/data')`) por nuevos puntos cada exactamente 1 segundo. Al recibirlos, actualiza el gráfico y los números de la pantalla.

Gestión del Reseteo (epoch): Revisa constantemente si el número de epoch que llega del servidor es distinto al que tenía guardado. Si cambia, entiende que el usuario presionó un botón de canal y vacía el arreglo de datos (`chart.data.datasets[0].data = []`) para limpiar la pantalla.

Comandos de Control: La función `select(sig)` está atada a los botones de la interfaz. Cuando se hace clic, envía una petición HTTP asíncrona (`fetch('/select?signal=X')`) para darle la orden al servidor Python de que cambie de señal.

Implementación:

Se transfieren los archivos `index.html` y `servidor.py` mediante el comando `scp`:

```
gaston@mint-virtual:~/SdeC/Practico_5$ scp -P 5022 index.html server.py root@localhost:
~/driver/
root@localhost's password:
index.html                               100% 6417    211.7KB/s   00:00
server.py                                100% 5005    307.3KB/s   00:00
gaston@mint-virtual:~/SdeC/Practico_5$
```

Figura 13: Transferencia de archivos `index.html` y `servidor.py`.

Luego, dentro de la VM se ejecuta:

```
python3 server.py --reader ./reader --dev /dev/seniales --port 8000
```

Entonces, al ir a la dirección “`http://localhost:8000`” en el navegador del host, se pueden visualizar las señales:



Figura 14: Señal Senoidal.

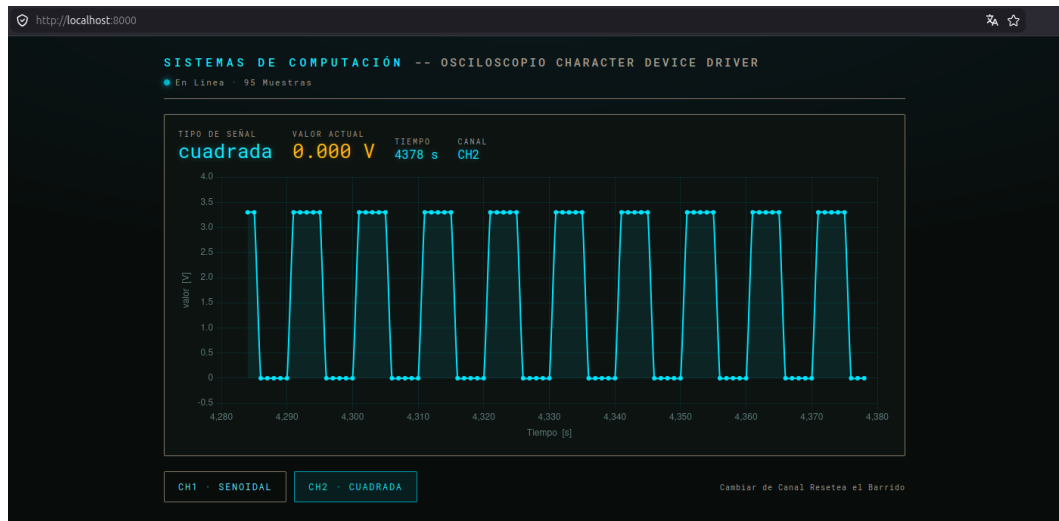


Figura 15: Señal Cuadrada.

3. Conclusiones

- En la parte de compilar el módulo `seniales.c`, no se utilizó compilación cruzada porque para ello es necesario tener el mismo kernel tanto en la raspberry simulada como en el host, de lo contrario el `vermagic` no coincide y no se puede compilar correctamente. Por otro lado, sí se realiza la compilación cruzada del archivo `reader.c`, y se verifica su correcto funcionamiento.
- El proyecto demuestra de forma práctica la división entre el espacio del núcleo (Kernel Space) y el espacio de usuario (User Space). Al generar los datos en el driver y procesarlos en un script externo, se asegura que un fallo en el servidor web o en el lector no provoque un kernel panic que congele todo el sistema.
- Se comprobó la enorme ventaja del diseño de Linux, donde un sensor o hardware complejo (`/dev/seniales`) puede ser operado desde el espacio de usuario usando las mismas operaciones que se usarían para leer un simple bloc de notas, abstrayendo toda la complejidad del hardware.
- Se evidencia cómo tecnologías de distintos niveles pueden cooperar de forma fluida. El lenguaje C maneja la velocidad y el bajo nivel, Python actúa como puente y JavaScript dibuja la interfaz. El uso de JSON como formato de intercambio fue clave para unificar estos tres mundos.
- El uso de QEMU permitió emular una arquitectura ARM64 completa. Esto demuestra que es posible diseñar, compilar y probar controladores de dispositivos sin poner en riesgo hardware físico real (como una Raspberry Pi) ni la computadora host, optimizando los ciclos de prueba y error.

4. Link al Repositorio

<https://github.com/GastonYomaha/SistemasDeComputacion/tree/main/TP5>